

# OPMIG-07: Metric & Event Extraction

---

**Series:** OPMIG — OpenPipeline Migration | **Notebook:** 7 of 10 | **Created:** December 2025 | **Last Updated:** 09/28/2026

---

## Table of Contents

---

1. [Metric Extraction](#)
  2. [SLI Metric Patterns: RED Methodology](#) ★ NEW
  3. [Event Extraction](#)
  4. [Business Event Extraction](#)
  5. [Business KPI Extraction](#) ★ NEW
  6. [Extraction Matching Conditions](#)
  7. [Practical Examples](#)
  8. [Validating Extractions](#)
  9. [Metric Design Best Practices](#) ★ NEW
  10. [Best Practices](#)
  11. [Complete Extraction Pipeline Example](#)
- 

## Learning Objectives

---

By completing this notebook, you will:

1. Configure metric and event extraction from logs
2. Generate business events for analytics
3. ★ **NEW:** Implement SLI metrics using RED methodology (Rate, Errors, Duration)
4. ★ **NEW:** Extract business KPIs (revenue, conversion, user journey)
5. ★ **NEW:** Apply metric design best practices (cardinality, naming, aggregation)

6. Validate extractions with DQL queries
7. Optimize extraction for cost efficiency

---

## Prerequisites

---

| Requirement                  | Details                                                                                                               |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Dynatrace Environment</b> | Dynatrace SaaS with Grail and OpenPipeline access — Managed is not covered by this series                             |
| <b>Permissions</b>           | <code>settings:read</code> , <code>settings:write</code> (OpenPipeline configuration),<br><code>metrics.ingest</code> |
| <b>API Access</b>            | <code>logs.read</code> , <code>logs.ingest</code> , <code>metrics.ingest</code> token scopes                          |
| <b>Knowledge</b>             | OPMIG-01 through OPMIG-06; understanding of processing/parsing concepts                                               |

---

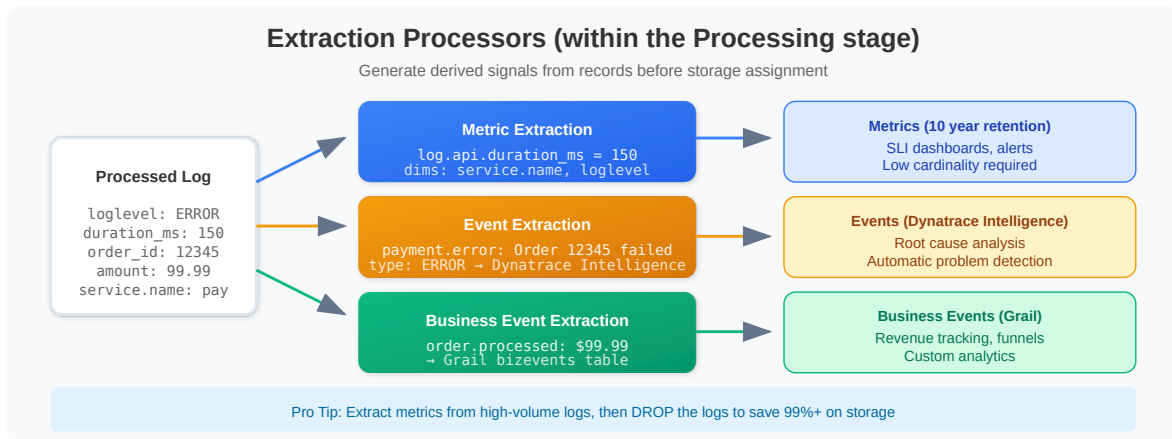
## Extraction Processors Overview

---

**Stage model:** Extraction runs in three dedicated stages after Bucket assignment — **Metric extraction**, **Events** and **Data extraction** (business events, SDLC events). Smartscape node and edge extraction are their own stages too, straight after Processing. The stage sequence is fixed; OPMIG-02 § *Understanding Processing Order* has the full table.

**Sources:** [Processing in OpenPipeline \(DT docs\)](#) — "The sequence of stages is fixed for all pipelines and cannot be modified."

The extraction processors generate derived signals (metrics, events, business events, Smartscape topology) from records before they reach Storage:



**Span sampling note:** When extracting metrics from spans, use the **sampling-aware** variants of counter/value/histogram processors so the extracted metric remains unbiased by trace sampling decisions. Span-specific extraction is covered in OPIPE-03 (sampling-aware metrics).

## Metric Extraction

OpenPipeline supports two types of metrics:

### Value Metrics (Gauge)

Extract numeric values as gauge metrics.

| Field              | Description                                          |
|--------------------|------------------------------------------------------|
| Key                | Metric name (e.g., <code>log.payment.amount</code> ) |
| Value              | Numeric field to extract                             |
| Dimensions         | Fields to use as metric dimensions                   |
| Matching Condition | When to apply extraction                             |

**Example: Extract payment amount**

```
Metric Type: Value
Key: log.payment.amount
Value: amount
Dimensions: service.name, environment, status
Matching: matchesPhrase(content, "payment") AND isNotNull(amount)
```

## Counter Metrics

Count occurrences of matching records.

| Field              | Description                                       |
|--------------------|---------------------------------------------------|
| Key                | Metric name (e.g., <code>log.error.count</code> ) |
| Dimensions         | Fields to use as metric dimensions                |
| Matching Condition | When to count                                     |

**Example: Count errors**

```
Metric Type: Counter
Key: log.errors.count
Dimensions: service.name, error_code
Matching: loglevel == "ERROR"
```

## Metric Naming Conventions

Format: `<source>.<domain>.<name>`;

Examples:

```
log.api.request_count
log.payment.amount
log.auth.failed_attempts
span.checkout.duration_ms
```

## Dimension Best Practices

| Do                        | Don't                   |
|---------------------------|-------------------------|
| Low-cardinality fields    | High-cardinality IDs    |
| <code>service.name</code> | <code>request_id</code> |

| Do                       | Don't                  |
|--------------------------|------------------------|
| <code>environment</code> | <code>user_id</code>   |
| <code>error_code</code>  | <code>timestamp</code> |

## SLI Metric Patterns: RED Methodology ★ NEW

The **RED Method** (Rate, Errors, Duration) provides comprehensive service observability.

### RED Overview

| Metric   | Measures                      | Why                      |
|----------|-------------------------------|--------------------------|
| Rate     | Requests/sec                  | Traffic volume, capacity |
| Errors   | Error rate (%)                | Reliability, SLA         |
| Duration | Response time (p50, p95, p99) | User experience          |

### Rate: Request Throughput

```
Type: Counter
Key: log.api.request_rate
Dimensions: method, path, status_category, service.name
Matching: isNotNull(path)
```

### Errors: Error Rate SLI

#### Total Requests:

```
Counter: log.api.request_total
Dimensions: service.name, endpoint
```

#### Error Requests:

```
Counter: log.api.request_errors
Dimensions: service.name, endpoint, error_type
Matching: status >= 500 OR loglevel == "ERROR"
```

## Duration: Latency Percentiles

```
Type: Value
Key: log.api.response_time_ms
Value: duration_ms
Dimensions: service.name, endpoint, status_category
```

## Latency Buckets (Apdex-style SLI)

Use `fieldsAdd` to create latency buckets:

```
fieldsAdd latency_bucket = if(duration_ms < 100, "fast",
                             else: if(duration_ms < 500, "acceptable",
                             else: if(duration_ms < 1000, "slow",
                             else: "very_slow"))
```

Then create a counter metric:

```
Counter: log.api.latency_bucket_count
Dimensions: latency_bucket, service.name, endpoint
```

## Event Extraction

Extract events that Dynatrace Intelligence can analyze for root cause analysis.

## Event Configuration

| Field      | Description                  |
|------------|------------------------------|
| Event Name | Unique event type identifier |

| Field              | Description                                           |
|--------------------|-------------------------------------------------------|
| Event Description  | Template for event description                        |
| Matching Condition | When to generate event                                |
| Event Type         | Category: INFO, AVAILABILITY, ERROR, RESOURCE, CUSTOM |

## Event Types

| Type         | Use Case                   |
|--------------|----------------------------|
| INFO         | Informational events       |
| AVAILABILITY | Service up/down events     |
| ERROR        | Error conditions           |
| RESOURCE     | Resource contention events |
| CUSTOM       | Custom event types         |

## Event Description Templates

Use field placeholders in descriptions:

```
Payment failed for order {order_id}: {error_message}
Service {service.name} error rate exceeded threshold
Authentication failure for user {user_id} from IP {client_ip}
```

## Example Event Extraction

```
Event Name: payment.failure
Event Description: Payment failed for order {order_id}: {error_message}
Event Type: ERROR
Matching: matchesPhrase(content, "payment") AND matchesPhrase(content, "failed")
```

## Attribution — what the extracted event attaches to

A Davis event only earns its place if Dynatrace can tell what it is *about*. Most Davis events carry `dt.smartscape_source.id`, the Smartscape entity ID of the signal's source —

84.9% on a validation tenant over 7 days, 08/11/2026 — and the documented grouping rule merges all events sharing that value into one problem. **An extracted event with the field unset is not exempt from that rule — it is attributed to the environment entity, so every such extraction merges into the same problem.** Since extraction fires per matching record, the result is not a flood of duplicate problems but a single problem of unbounded scope: unrelated log conditions from across the estate welded together, with no root cause and nothing to route on.

**Extraction reads the record; it cannot resolve an entity that is not on it.** A log OneAgent collected from a monitored process arrives already carrying `dt.entity.host`, `dt.entity.process_group_instance`, and on Kubernetes `dt.entity.cloud_application`. A log pushed through the generic ingest API or forwarded by a third-party shipper frequently arrives with none of that, and no extraction rule can conjure it. Enrichment is therefore a **prerequisite** of event extraction, not a nice-to-have alongside it.

Check the source stream before you configure the extraction, not after:

```
// Entity-enrichment coverage on the stream you plan to extract events from.
// Low coverage here means extracted Davis events will be unattributable.
fetch logs, from:-2h
| summarize total = count(),
    with_host = countIf(isNotNull(dt.entity.host)),
    with_pgi = countIf(isNotNull(dt.entity.process_group_instance)),
    with_workload = countIf(isNotNull(dt.entity.cloud_application))
| fieldsAdd host_pct = round(with_host * 100.0 / total, decimals: 1),
    pgi_pct = round(with_pgi * 100.0 / total, decimals: 1),
    workload_pct = round(with_workload * 100.0 / total, decimals: 1)
```

Executed on a validation tenant over 2 hours, 07/31/2026: **1,369,261 records at 97.8% host, 97.8% process-group, and 97.7% workload coverage** — the shape you expect where OneAgent collects nearly everything. The residual couple of percent is precisely where an extracted event would land unattributed.

If your own figure is materially lower, the streams behind the gap are usually API-pushed or forwarder-sourced. Fix enrichment there first — OPLOGS-03 §4 covers attribute creation, and its OneAgent attribute-enrichment note covers setting primary fields at source — rather than extracting events that cannot correlate. Filter this query by `dt.openpipeline.source` or `k8s.namespace.name` to isolate the specific stream your extraction rule matches.

Once extraction is live, confirm the events actually carry a key: AIOPS-02 §8 ranks detectors and extracted event sources by firing count and shows which are unattributed, and AIOPS-03 §1 covers the correlation rules themselves.

**Business events are a different case entirely** — the next section. They never open problems, so none of this applies to them.

**Sources:** [Avoid overalerting \(DT docs\)](#) — "make sure to set the `dt.smartscope_source.id` field to an existing Smartscape entity ID". **Derived:** treating source-record enrichment as a prerequisite of event extraction follows from the correlation requirement plus the fact that an extraction rule can only read fields present on the record.

## Business Event Extraction

Business events are stored in Grail for business analytics.

### Business Event Configuration

| Field              | Description                |
|--------------------|----------------------------|
| Event Type         | Business event type name   |
| Event Provider     | Source system identifier   |
| Data Fields        | Fields to include in event |
| Matching Condition | When to extract            |

### Example: Order Events

```
Event Type: com.example.order.placed
Event Provider: checkout-service
Data Fields:
  - order_id
  - customer_id
  - total_amount
  - currency
  - timestamp
Matching: matchesPhrase(content, "order placed successfully")
```

## Querying Business Events

---

## Business KPI Extraction NEW

---

### Revenue Tracking

#### Order Revenue:

```
Type: Value
Key: log.revenue.order_amount
Value: amount
Dimensions: currency, service.name, payment_method
```

### Conversion Funnels

#### E-Commerce Funnel:

1. Product View → 2. Add to Cart → 3. Checkout → 4. Complete

Use `fieldsAdd` to categorize funnel stages:

```
fieldsAdd funnel_stage = if(action == "product_view", "1_view",
                             else: if(action == "add_to_cart", "2_cart",
                             else: if(action == "checkout_started", "3_checkout",
                             else: "4_complete")))
```

### Feature Usage

```
Counter: log.feature.usage_count
Dimensions: feature_name, feature_enabled, user_segment
```

---

```
// Query business events extracted from logs
fetch bizevents, from: now() - 24h
| filter event.type == "com.example.order.placed"
| fields timestamp, order_id, customer_id, total_amount
| limit 50
```

```
// Business event analytics - order totals by hour
fetch bizevents, from: now() - 24h
| filter event.type == "com.example.order.placed"
| makeTimeseries {
  order_count = count(),
  total_revenue = sum(total_amount)
}, interval: 1h
```

---

## Extraction Matching Conditions

---

Matching conditions determine which records trigger extraction.

### Common Patterns

| Pattern           | Condition                                                                                                                                                                          |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Error logs        | <code>loglevel == "ERROR"</code>                                                                                                                                                   |
| Specific service  | <code>service.name == "payment-service"</code>                                                                                                                                     |
| Contains text     | <code>matchesPhrase(content, "failed")</code> (whole token) or <code>matchesValue(content, "*failed*")</code> (any substring) — <code>contains()</code> is not enabled in matchers |
| Has field         | <code>isNotNull(order_id)</code>                                                                                                                                                   |
| Numeric threshold | <code>duration_ms &gt; 1000</code>                                                                                                                                                 |

## Combining Conditions

```
// AND - both must match
loglevel == "ERROR" AND service.name == "payment"

// OR - either matches
status == "failed" OR status == "error"

// Complex conditions
(loglevel == "ERROR" OR loglevel == "WARN")
AND matchesPhrase(content, "payment")
AND isNotNull(amount)
```

## Field Reference in Conditions

Use any field available after processing:

- Original fields from ingestion
- Parsed fields from processors
- Computed fields from `fieldsAdd`

## Practical Examples

### Example 1: API Request Metrics

Source Log:

```
2024-12-12T10:30:45 INFO - GET /api/users completed in 150ms, status=200
```

Pipeline Processing:

```
// First, parse the log
parse content, "LD:method ' ' LD:path ' completed in ' INT:duration_ms 'ms,
status=' INT:status_code"
```

Metric Extraction 1: Request duration

```
Type: Value
Key: log.api.request_duration_ms
Value: duration_ms
Dimensions: method, path, status_code
Matching: isNotNull(duration_ms)
```

## Metric Extraction 2: Request count

```
Type: Counter
Key: log.api.request_count
Dimensions: method, path, status_code
Matching: isNotNull(path)
```

## Example 2: Error Event Generation

### Source Log:

```
2024-12-12T10:30:45 ERROR PaymentService - Transaction failed: insufficient
funds, orderId=12345
```

### Event Extraction:

```
Event Name: payment.transaction.failed
Event Description: Transaction failed for order {order_id}: {error_reason}
Event Type: ERROR
Matching: loglevel == "ERROR" AND matchesPhrase(content, "Transaction failed")
```

## Example 3: Login Business Events

### Source Log:

```
2024-12-12T10:30:45 INFO AuthService - User login successful: userId=john123,
ip=192.168.1.100
```

### Business Event Extraction:

```
Event Type: com.example.auth.login
Event Provider: auth-service
Data Fields: user_id, client_ip, timestamp
Matching: matchesPhrase(content, "login successful")
```

## Example 4: Combined Extraction

From a single payment log line, extract:

1. **Metric:** `log.payment.amount` (value: amount)
  2. **Metric:** `log.payment.count` (counter)
  3. **Event:** Payment processed (INFO)
  4. **Business Event:** Order transaction
- 

## Validating Extractions

After configuring extractions, verify they're working.

**List log-extracted metrics:** browse them in the Dynatrace UI (**Observe > Metrics**), or query a specific metric with `timeseries avg_value = avg(log.your_metric_name), from: now() - 24h`.

```
// Query a specific extracted metric
// Replace 'log.api.request_duration_ms' with your metric key
timeseries {
  avg_duration = avg(log.api.request_duration_ms),
  max_duration = max(log.api.request_duration_ms)
}, from: now() - 2h, interval: 5m
```

```
// View extracted counter metric by dimension
// Replace 'log.api.request_count' with your metric key
timeseries {
  requests = sum(log.api.request_count)
}, from: now() - 2h, interval: 5m
```

```
// View extracted events
fetch events, from: now() - 24h
| filter isNotNull(event.name)
| summarize {event_count = count()}, by: {event.type, event.name}
| sort event_count desc
```

```
// View specific event details
fetch events, from: now() - 24h
| filter event.name == "payment.transaction.failed"
| fields timestamp, event.name, event.description, event.type
| sort timestamp desc
| limit 25
```

```
// View business events by type
fetch bizevents, from: now() - 24h
| summarize {bizevent_count = count()}, by: {event.type, event.provider}
| sort bizevent_count desc
```

```
// Compare source log volume to extracted metrics
// This helps verify extraction is capturing expected data
fetch logs, from: now() - 1h
| filter contains(content, "payment")
| summarize {log_count = count()}
| fieldsAdd data_source = "logs"

// Then compare with:
// timeseries sum(log.payment.count)
```

---

## Metric Design Best Practices NEW

---

### Cardinality Management

| Cardinality | Examples               | Max Values | Use?    |
|-------------|------------------------|------------|---------|
| Low         | environment, region    | < 50       | ✓ Ideal |
| Medium      | endpoint, error_code   | 50-500     | ✓ Good  |
| High        | session_id, request_id | 500-10K    | ⚠ Avoid |
| Very High   | user_id, order_id      | > 10K      | ✗ Never |

**Cardinality Example:**

- ✓ GOOD: 5 services × 3 environments × 4 methods = 60 time series
- ✗ BAD: 5 services × 100K user\_ids = 500K time series

## Reduction Techniques:

```
// Bucket high-cardinality values
fieldsAdd duration_bucket = if(duration_ms < 100, "<100ms",
                                else: if(duration_ms < 500, "100-500ms",
                                else: ">500ms"))

// Use segments instead of IDs
fieldsAdd user_segment = if(user_tier == "premium", "premium", else: "free")
```

## Naming Conventions

**Format:** <source>.<domain>.<metric>.<unit>

### Examples:

- ✓ log.api.request\_count
- ✓ log.api.response\_time\_ms
- ✓ log.payment.order\_amount\_usd
- ✓ log.auth.failed\_login\_count
- ✗ apiReqCnt (abbreviated, no unit)
- ✗ response\_time (no source, no unit)
- ✗ count (no context)

## Cost Optimization

### Metrics vs. Logs:

Scenario: 1M requests/day

Option 1: Store logs (35 days)

- 35M log records × 500 bytes = 17.5 GB
- Cost: ~\$140/month

Option 2: Extract metrics + No storage assignment for the logs

- 10 time series, 10 years retention = ~10 MB
- Cost: ~\$1/month
- Savings: 99.3% 🎉

### When to Extract:

-  SLI tracking (RED)
  -  Business KPIs
  -  High-volume health checks
  -  Debugging (keep logs)
  -  Compliance/audit (keep logs)
- 
- 

## Best Practices

---

### Metric Extraction

| Practice                        | Reason                             |
|---------------------------------|------------------------------------|
| Use meaningful metric names     | Easier discovery and querying      |
| Limit dimensions ( $\leq 5-7$ ) | Avoid metric cardinality explosion |
| Use low-cardinality dimensions  | IDs cause metric bloat             |
| Extract only what you need      | Reduces storage costs              |
| Test matching conditions        | Ensure correct data selection      |

### Event Extraction

| Practice                | Reason                                   |
|-------------------------|------------------------------------------|
| Meaningful event names  | Easier to find in Dynatrace Intelligence |
| Descriptive templates   | Context for troubleshooting              |
| Appropriate event types | Correct Dynatrace Intelligence handling  |
| Avoid over-extraction   | Too many events = noise                  |

## Business Event Extraction

| Practice                  | Reason               |
|---------------------------|----------------------|
| Consistent event types    | Enables analytics    |
| Include all needed fields | Avoids re-processing |
| Use namespaced types      | Prevents collisions  |
| Document event schema     | Helps consumers      |

## Cost Optimization Pattern

**Extract → Don't store → Save**

1. Extract metrics and events from verbose logs
2. Assign the original verbose logs **No storage assignment** (Bucket assignment stage)  
— the record still runs through the extraction stages but is not stored
3. Keep only derived signals

Do not use a **Drop record** processor for step 2: it runs in the Processing stage, before any extraction stage, so nothing would be extracted.

**Sources:** [Processing in OpenPipeline \(DT docs\)](#) — Drop record: "Drops a record. The record isn't processed further and isn't stored."; No storage assignment: "The record continues through all configured pipeline stages and isn't stored only at the end of the pipeline."

This pattern reduces storage while preserving observability.

## Complete Extraction Pipeline Example

**Pipeline:** `checkout-service-logs`

**Processing Stage (from OPMIG-06):**

```
// Parse order details
parse content, "'orderId=' INT:order_id ','"
| parse content, "'amount=' DOUBLE:amount"
| parse content, "'status=' LD:order_status"
| parse content, "'duration=' INT:duration_ms 'ms'"
```

### Metric Extraction 1: Order Amount

```
Type: Value
Key: log.checkout.order_amount
Value: amount
Dimensions: order_status, service.name
Matching: isNotNull(amount)
```

### Metric Extraction 2: Order Count

```
Type: Counter
Key: log.checkout.order_count
Dimensions: order_status, service.name
Matching: isNotNull(order_id)
```

### Metric Extraction 3: Processing Duration

```
Type: Value
Key: log.checkout.processing_duration_ms
Value: duration_ms
Dimensions: order_status, service.name
Matching: isNotNull(duration_ms)
```

### Event Extraction: Order Failure

```
Event Name: checkout.order.failed
Event Description: Order {order_id} failed with status {order_status}
Event Type: ERROR
Matching: order_status == "failed" OR order_status == "error"
```

### Business Event: Order Completed

```
Event Type: com.example.checkout.order_completed
Event Provider: checkout-service
Data Fields: order_id, amount, order_status, duration_ms
Matching: order_status == "completed" OR order_status == "success"
```

---

# Next Steps

---

Now that you can extract metrics and events, continue with:

| Notebook | Focus Area                     |
|----------|--------------------------------|
| OPMIG-08 | Security, Masking & Compliance |
| OPMIG-09 | Troubleshooting & Validation   |

---

# References

---

- [OpenPipeline Data Extraction](#)
  - [Extract metrics from spans and distributed traces \(DT docs\)](#)
  - [OpenPipeline use cases \(DT docs\)](#)
  - [Dynatrace Intelligence Events](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*